Chiron

Final Design Consolidation and Harness-First Delivery Alignment

Progress Report 12 • 1 Mar 2026 — 14 Mar 2026

Master’s Research Progress Update

Fahad Alothman

Progress Report 12

Title: Final Design Consolidation and Harness-First Delivery Alignment
Reporting Window: 1 Mar 2026 — 14 Mar 2026
Submission Context: Master’s Research Progress Update

Abstract

This reporting window can be read as the clearest summary of Chiron’s current design direction. The project did not merely continue implementation. It clarified what the system is prepared to keep as core, what should be deferred until the foundations are stronger, and what should be rejected as the wrong product shape. The period therefore serves as both a runtime formalization phase and a practical design statement.

Two ideas dominate the window. First, methodology definitions became more legibly connected to runtime records: facts, transitions, workflows, and artifacts now map more clearly from authored contracts to executed evidence. Second, the product direction aligned around a harness-first strategy for practical local execution, while deeper AX- and AI-SDK-heavy ambitions were intentionally held back. Together, these decisions explain the current Chiron feature set more accurately than any single lower-level implementation detail.

PR12 as a Summary of What Chiron Is Becoming

By this stage, Chiron is best understood as a methodology-authoring and methodology-execution system. It is not only a workflow editor, not simply an agent shell, and not primarily a runtime for abstract AI orchestration. Its core value lies in binding authored methodology to durable project execution.

That means the central structure is no longer in doubt. Chiron keeps a layered design in which L1 defines methodology-wide topology and reusable registries, L2 defines the contract of a work unit, and L3 defines workflow grammar and step-level execution structure. It also keeps a mirrored split between Methodology Design Time and Project Runtime. Design-time defines what work means, what transitions are allowed, what facts and artifacts matter, and what workflows can run. Runtime records what actually happened under those authored conditions.

The significance of PR12 is that this model stops reading like a future possibility and starts reading like the current product shape.

What Was Formalized in This Window

The most important formalization work in this period was the strengthening of the model-to-runtime chain. Facts defined at methodology or work-unit level map to runtime fact instances. Transition policy maps to transition executions. Workflow definitions map to workflow executions. Context facts map to runtime context values where workflow execution requires them. Artifact-slot definitions map to runtime artifact evidence.

This is especially important because it clarifies that runtime is not supposed to invent its own semantic layer. Runtime exists to execute and record the methodology. That is why version pinning, execution status, lineage, and evidence all matter so much. Chiron's runtime is becoming more useful precisely because it is becoming more constrained by authored policy.

Artifact behavior offers the clearest example. The important slot distinction is not a large rule system but the simpler contract between `single` and `fileset`. On the runtime side, the slot is not represented by one mutable "current artifact" row. It is represented by immutable snapshots and file-member evidence. Those members can carry commit/blob references, snapshots can supersede previous snapshots, and current-state checks can determine whether effective state is changed, unchanged, or unavailable. That turns artifact outputs into durable evidence that gates, downstream workflows, and review surfaces can reason about instead of treating them as informal side effects.

Why Harness-First Became the Practical Direction

This period also made the execution strategy more realistic. The project had already explored broader AI-runtime directions, including deeper AX- and AI-SDK-heavy expansion. However, the more important question at this stage was not what was theoretically ambitious. It was what would produce a coherent and usable system for actual users, especially local individual users.

That is why harness-first execution became the preferred direction. Harness-oriented integrations already provided usable execution surfaces and simpler provider pathways, while deeper AX and AI-SDK-centric expansion remained partially scaffolded. Building the product around those unfinished layers would have produced more conceptual complexity without improving the durability of the runtime model. The decision therefore became to preserve the stronger practical baseline: use harnesses now, keep methodology and runtime evidence deterministic, and defer broader orchestration ambition until the core system is finished enough to support it safely. The MCP-facing side of that boundary also became clearer through the move toward Hono-based MCP middleware, Hono itself, and the Model Context Protocol SDK [7]–[9].

External comparisons reinforced that direction. OpenCode, Codex, and PI all demonstrate forms of harness- or CLI-oriented execution that suit local or self-managed workflows better than prematurely abstracted orchestration layers [2]–[4]. Chiron's design direction in this window is consistent with that lesson, but filtered through its own methodology-first needs.

Final-Design Summary: What Chiron Keeps

By the end of this window, a relatively stable "kept" set is visible.

Chiron keeps its identity as a methodology-first system. It keeps the L1/L2/L3 layered model. It keeps the separation between Methodology Design Time and Project Runtime. It keeps methodology version pinning as the bridge between authored policy and project execution. It keeps methodology/project facts, work-unit facts, and context facts as distinct systems rather than collapsing them into a single fact abstraction.

It also keeps methodology-defined dependency and link semantics as data rather than engine enums. It keeps transitions and gates as lifecycle authority. It keeps workflows attached to work-unit contracts rather than floating independently. It keeps artifact slots as durable output contracts and runtime artifact snapshots as evidence with lineage and git-aware change detection. Most importantly, it keeps the rule that durable workflow-to-workflow business data should move through persisted facts and artifact slots rather than generic invoke payload mapping. The intended repository/workspace story remains consistent with that same discipline: git-aware truth is explicit, and simple-git remains part of the intended operations layer rather than hidden infrastructure magic [12].

At the product level, it keeps harness-first execution as the practical baseline for current delivery.

Final-Design Summary: What Chiron Defers

The deferred set is equally important because it explains why the current feature set is narrower than the total imagination space around the project.

Chiron defers deeper AX and AI-SDK-heavy runtime expansion as a baseline dependency. It defers broader AX- and AI-SDK-heavy runtime expansion beyond the harness-first baseline. It defers universal end-to-end step-runtime completion across all workflow behavior. It defers broader advanced runtime ergonomics that would increase surface complexity before the contract model is fully settled. It defers abstract output-mapping systems around invoke, because those would compete with the durable facts-and-artifacts model rather than strengthen it.

More generally, the system defers any feature that would outrun deterministic persistence, lineage, gate semantics, and runtime evidence.

Final-Design Summary: What Chiron Rejects

Some directions are not merely postponed. They are increasingly inconsistent with the product shape established here.

Chiron rejects the idea that it should be only a workflow builder. It rejects the idea that invoke should act as a business IO mapping layer. It rejects copied child payloads as a preferred way to move durable business results. It rejects hardcoded dependency meaning inside runtime enums when methodology data can carry that meaning instead. It rejects mutable “latest artifact” records without lineage. It rejects treating outputs as informal side effects instead of contract-bound evidence. Finally, it rejects allowing unfinished AI infrastructure to define the product’s identity before the methodology and runtime model are strong enough to support that ambition.

These rejected directions are as important as the kept ones, because they explain why the current feature set looks disciplined rather than sprawling.

Why This Window Matters

PR12 matters because it converts the previous two windows into a more stable design statement. PR10 made the architecture coherent. PR11 made it harder to violate. PR12 explains what this now means for the actual product: a methodology-first system with layered contracts, durable runtime evidence, git-aware artifacts, and a harness-first execution strategy.

This is also the point where the feature set becomes more understandable as a deliberate cut. What remains is not everything the project could eventually include. It is the subset that best reinforces determinism, inspectability, auditability, and practical local execution.

That deliberate cut should not be misread as indifference to isolation or orchestration. Broader git-worktree control, deeper sandboxing, and richer agent coordination remain part of the longer-horizon Chiron direction. They simply no longer define the master’s-project core. In the current shape, git is used chiefly where it sharpens durable project evidence — especially around artifact truth, lineage, and staleness — while the thesis-critical implementation priority is the guidance and shared project model that both users and agents execute against.

Stack Direction at the End of the Window

The technical stack at this point remains consistent with the direction established earlier:

- **Desktop host:** Tauri [5]
- **Frontend:** React + TanStack Router + shadcn/ui
- **Backend/API:** TypeScript services with Hono + oRPC-aligned contracts, plus MCP transport direction through Hono-based MCP middleware, Hono itself, and the Model Context Protocol SDK [6]–[8]
- **Execution/control plane:** Effect-first with harness-first runtime alignment, with `HarnessService` / `OpencodeHarnessService` and the OpenCode SDK defining the current agent-runtime boundary [1], [2]
- **Data layer:** Drizzle ORM with methodology-first persistence and stronger runtime lineage on a SQLite-first, libSQL/Turso-compatible baseline [9]
- **Git/workspace seam:** shared git-truth query direction through sandbox-engine and `GitWorkspaceQueryService`, with simple-git still part of the intended repository/workspace operations story [10]
- **Authentication:** Better Auth [11]
- **Runtime/tooling:** Bun + Turborepo [12]

The important message is that the stack now serves a clearer product shape rather than an open-ended experimentation space. The SQLite decision is part of that. The project narrowed from broader database ambitions to a SQLite-first current horizon because Chiron's immediate value depends on local execution, lower operational friction, and deterministic methodology/runtime behavior more than on early distributed scale. Keeping the stack libSQL/Turso-compatible preserves a later hosted path without forcing the present design to optimize for a second operating mode too early.

Conclusion

Progress Report 12 serves as the clearest current summary of Chiron's design. The system keeps methodology-first structure, layered ownership, durable facts and artifacts, explicit lifecycle authority, and harness-first practical execution. It defers broader AI-runtime ambition until the core execution model is stronger. It rejects product directions that would weaken determinism or hide meaning inside ad hoc workflow plumbing.

In that sense, this window is not merely another implementation phase. It is the point where Chiron's current feature set and product identity become much easier to state plainly.

References

[1] Effect Authors, “Effect Documentation,” 2026. [Online]. Available: https://effect.website/

[2] OpenCode, “OpenCode Documentation,” 2026. [Online]. Available: https://opencode.ai/

[3] OpenAI, “Codex,” GitHub repository, 2026. [Online]. Available: https://github.com/openai/codex

[4] badlogic, “pi-mono,” GitHub repository, 2026. [Online]. Available: https://github.com/badlogic/pi-mono

[5] Tauri Programme, “Tauri,” Documentation, 2026. [Online]. Available: https://tauri.app/

[6] Hono Authors, “Hono,” Documentation, 2026. [Online]. Available: https://hono.dev/

[7] Hono, “@hono/mcp,” npm, 2026. [Online]. Available: https://www.npmjs.com/package/@hono/mcp

[8] Model Context Protocol, “Introduction,” 2026. [Online]. Available: https://modelcontextprotocol.io/docs/getting-started/intro

[9] Drizzle Team, “Drizzle ORM Documentation,” 2026. [Online]. Available: https://orm.drizzle.team/

[10] simple-git, “simple-git,” GitHub repository, 2026. [Online]. Available: https://github.com/simple-git-js/simple-git/

[11] Better Auth, “Better Auth Documentation,” 2026. [Online]. Available: https://www.better-auth.com/docs

[12] Turborepo Authors, “Turborepo Documentation,” 2026. [Online]. Available: https://turbo.build/repo/docs